



MOBILE SOFTWARE DEVELOPMENT

- COMP4310 -

App Architecture: Separation of Concerns (MVVM)

CineVault: Movie Discovery App

Team Members:

Walaa Jaradat	1220198
Tuleen Rimawi	1220211
Masa Inabi	1231024
Aya Jiaan	1232406
Mona Omar	1220051

Instructor : Dr. Faisal Shehadeh

GitHub Repository Link: <https://github.com/walaaJaradat/Mobile-project.git>

Table of Contents

Table of Contents	I
Table of Figures.....	II
Executive Summary	3
1. Introduction & Problem Statement	4
1.1 Background:.....	4
1.2 The Problem.....	4
1.3 Scope of Research	5
2. Technical Framework & Under-the-Hood Architecture	5
2.1 Core Concepts:.....	5
3 Prototype Implementation & Engineering Design	8
3.1 Prototype Overview	8
3.2 App Architecture & Design Patterns	9
3.3 Key Code Snippets & Breakdown:	10
4 Discussion and Conclusion	12
4.1 Summary of Findings.....	12
4.2 Lessons Learned.....	13
4.3 Team Contributions.....	14
5. References.....	15

Table of Figures:

Figure 1:CineVault MVVM architecture.....	10
Figure 2:MovieViewModel search and filtering logic.	11
Figure 3:MovieRepository data-access boundary	12

Executive Summary

This report analyzes the Model-View-ViewModel (MVVM) architectural pattern as a practical solution for separating responsibilities in modern mobile application development. The technical problem addressed by MVVM is the common tendency to place user-interface code, state management, business rules, and data-access operations inside the same Activity or screen file. That approach may work for a small prototype, but it quickly becomes hard to maintain, difficult to test, and risky to extend as an application grows.

CineVault demonstrates MVVM through a movie discovery application built on Android using Java, XML layouts, Activities, ViewModel, LiveData, the Repository Pattern, local movie data, and a prepared Retrofit networking layer for future API integration. The application supports movie browsing, search, filtering by genre/year, movie details, and favorites. A small Flutter prototype using Dart and local/static movie data is also included as a comparison point for state-management ideas across platforms. The main focus, however, remains the Android MVVM implementation.

This report highlights how MVVM is an excellent match for the mobile app ecosystem of 2026 due to increased need of lifecycle awareness in state management, testability of business logic, reactive UI, and ability of data layers to switch between local and remote databases. The view component is responsible for displaying UI and managing user interaction, ViewModel provides UI state using LiveData, and the repository component manages local movie database with room for future Retrofit integration.

1. Introduction & Problem Statement

1.1 Background:

Mobile applications are now expected to be responsive, reliable, and easy to extend. As applications grow, developers must manage screen state, user events, filtering rules, navigation, local data, and possible network data without allowing the codebase to become tightly coupled. In traditional Android development, a common problem is placing too much code inside an Activity: UI rendering, click handling, filtering, data retrieval, and state preservation may all be mixed together. This creates large classes that are hard to read, hard to test, and hard to modify safely. (Android Developers, 2026)

This issue will be addressed using the MVVM pattern, which means that the application is going to be split into different layers with certain tasks assigned to each of them. In the View layer, there needs to be a responsibility for managing the components and receiving user input. ViewModel will have to manage the states of the UI in a lifecycle-aware manner. There is going to be another layer – the Model and the Repository responsible for the data management. (Android Developers, 2026)

1.2 The Problem:

MVVM architecture is critical for Android development due to the following reasons:

- **Separation of concerns:** an Activity must render UI elements and dispatch events rather than implement the full filtering and data access logic (Torunoglu, 2024).
- **Maintainability:** the modifications regarding user interface, filtering, and data source can be made in their respective layers without affecting other pieces of the code (Torunoglu, 2024).
- **Lifecycle management:** ViewModel retains the state of UI elements through lifecycle events such as rotation of the device (Torunoglu, 2024).
- **Testability:** testing the filtering and state preparation logic can be performed using ViewModel in isolation from the rest of the Android UI environment (Torunoglu, 2024).
- **Scalability:** implementation of the Repository pattern allows an app to move from dealing with local data to communicating with an imagined API service without altering the activity layer (Torunoglu, 2024).
- **Reactive updates:** the LiveData allows for sending notifications to the View on any modification of the movie list or filtering criteria (Torunoglu, 2024).

1.3 Scope of Research:

In this report, the use and evaluation of MVVM for mobile app development is discussed using the CineVault sample app. The report analyzes the effect of MVVM on the aspect of maintainability, separation of concerns, lifecycle management, and extendibility vis-a-vis scenarios in which UI and business logic code are intermixed (Google, 2026).

The Android prototype is implemented using Java, XML layouts, Activities, ViewModel, LiveData, a Repository, local movie data, and a prepared Retrofit networking layer. The current application uses local data for reliable testing of search and filtering behavior, while the Retrofit layer is included as future-ready infrastructure for backend/API integration (Google, 2026).

A simplified Flutter prototype is included only as a comparison of state management concepts. It uses Dart and local/static movie data to show how Flutter can express similar separation between UI widgets and state. The comparison supports the research discussion but does not replace the main Android MVVM analysis (Google, 2026).

The scope will include architecture, state management, local database operations, searching/filters, movie details, favorites, and preparing for future API capabilities. However, the prototype will not include user authentication, cloud syncing, recommendation logic, payment options, or any other features that require a backend system.

2. Technical Framework & Under-the-Hood Architecture

2.1 Core Concepts:

MVVM is an architectural approach where an application on mobile devices is broken down into three major parts - Model, View, and ViewModel. (Microsoft, 2012) It is quite popular in the Android world where this approach is achieved with the help of lifecycle aware architecture components like ViewModel and LiveData. The point is not to make code elegant but to control its flow within the app. (Android Developers, 2026)

- Model: represents data objects and operations in relation to data. In the case of CineVault, movie objects and paths for accessing data are part of this component. (Android Developers, 2026)

- **View:** is responsible for representing the UI layer. CineVault uses Activities and XML layouts that show lists of movies, searches, filters, details, and options to add favorites. (Android Developers, 2026)

- **ViewModel:** represents the state of screens and is used to receive event triggers from the View. It handles logic on the UI level, e.g., filtration, and provides observable results by means of LiveData. (Android Developers, 2026)

- **Repository:** goes between the ViewModel and data sources and does not care whether the data comes from local movies or a Retrofit API in the future. (Fowler, 2004)

Repository Pattern

This pattern was used in the development of CineVault such that the ViewModel class will not work directly with the data source but rather will use the Repository. (Fowler, 2004) It will then be decided how the data should be sourced by the repository. For now, the repository returns movies data locally in order to test stably. Future implementation will involve the fetching of data from an online API using Retrofit library. This means that changes made to the data source will affect the Activity and ViewModel classes minimally.

This means that even by 2026, MVVM would still have to operate within the mobile context because of the need for a reactive UI, offline/local-first data, APIs, automation testing, and updates. This way, developers could easily meet these new demands using such architecture. In relation to CineVault, developers would be able to develop movies locally while remaining integrable later.

ViewModel:

ViewModel is basically a life-cycle aware component used for storing and processing data to display on the UI. What goes behind the scene here is that, when an activity needs a ViewModel, the Android framework uses a ViewModelProvider, which will use a ViewModelStore provided by the corresponding Activity or Fragment. This way, the ViewModel created will not be destroyed during screen rotation (Trivedi, 2024).

The ViewModel should have no direct connections with activities, views or widgets used on the user interface. This is because the use of such connections could cause memory leak issues or even the destruction of the screen. The ViewModel holds data which can be observed by the activity. (Developers, 2024)

The CineVault ViewModel workflow is:

1. The Activity receives a user action, such as typing a search query or selecting a filter.
2. Data or event is being transmitted from the Activity class to the ViewModel.
3. The movie's information is being retrieved from the repository through the ViewModel using the state-driven UI approach.
4. There will be changes in LiveData whenever the movies and their details are filtered.
5. Changes to the LiveData can be monitored in the Activity without performing any operation for updating the RecyclerView data.

Search and filtering functionality has to be included in the ViewModel and not in the Activity as it defines screen states. While it is important for the Activity to know how to display a search and filter option, it is not important for it to decide which movies fit with the given details. By implementing the above design, it will guarantee that the CineVault application will have a simple interface, easily testable filtering functionality, and persistence of filter selections despite configuration changes. Furthermore, this will eliminate any duplicity of filtering code as this state can also be utilized by other Activities.

Live Data

LiveData is a lifecycle-aware observable data holder provided by Android Architecture Components. It stores UI-related data and notifies active observers when that data changes. In an Android Activity, this allows the UI to react to changes without repeatedly asking the ViewModel for updates (Google, 2026).

LiveData follows the observer pattern. In case of CineVault, the Activity would be the observer while the list of movies provided by the ViewModel would be the observable

list. As soon as the user searches for something new or uses the filters based on genre, year, and ViewModel creates the list based on filters applied (Gamma, Helm, Johnson, & Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, 1994).

LiveData is very appropriate for our prototyping since it fits well into the lifecycle of Android. This implies that it is only going to inform those observers with an active lifecycle. Consequently, there will be no wastage of computing power on inactive screens.

Retrofit

Retrofit is a type-safe HTTP client commonly used in Android applications to communicate with REST APIs. It allows developers to define API requests through Java interfaces and annotations such as GET and Query, while a converter such as Gson can map JSON responses into Java model objects (Square, n.d.).

Currently, within the prototype of CineVault app, movies are stored in local storage, in order to test searches, filters, details, and favorites regardless of internet connection and without the need for any API key. However, as part of this assignment, the project involves an already developed Retrofit networking layer in form of RetrofitClient and API service interfaces (Square, n.d.).

This design keeps the networking concern outside the Activity. If a remote API is added later, the Repository can become responsible for deciding whether data should come from the local source or Retrofit. The ViewModel and Activity should not need major changes because they already depend on the Repository rather than directly on the data source (Square, n.d.).

3 Prototype Implementation & Engineering Design

3.1 Prototype Overview:

- CineVault is a movie discovery app developed using the MVVM architecture.

- For the Android version, development was done using Java programming language, XML layout files, Activities, ViewModel, LiveData, Repository Pattern, movie data, and Retrofit layer set up for future implementation of APIs.
- Functionalities that it supports include browsing movies, searching for movies based on their title, filtering movies according to genre/year, movie detail, and managing favorites.
- This flutter prototype can be considered a mini comparative prototype that uses Dart as its programming language and static movie data to compare different states management.

3.2 App Architecture & Design Patterns:

Architectural Overview

The CineVault uses MVVM architectural pattern wherein the separation is done between the interface, the state of screens, and the data accessing. The View is composed of Android activities and also the XML layouts. This part contains the movie list, filters, detail screen, and favorites actions. The ViewModel has the state of the UI which involves the search input, filters, movie list according to the filters, and selected movie information. Lastly, the Repository handles the data access throughout the app and hides the difference between the current local data and future remote API data.

The Three Layers

Model layer: holds movie data and operations related to the movie data. In this prototype, the data will be local, whereas Retrofit will be ready to use API at later times.

View layer: includes activities and layouts written using XML code. View will render the UI, collect input from users and observe LiveData of the ViewModel.

ViewModel layer: will receive events from View layer, apply any UI-related operation like searching and filtering, and will emit state via LiveData.

Integration with Repository Pattern

Repository pattern enhances MVVM architecture by introducing an explicit boundary between ViewModel and the data source. ViewModel will not have to care about data coming from hardcoded sources, files or Retrofit in the future. This will increase modularity because any modifications in data sources in the future will be restricted mostly to Repository and networking layers.

Configuration Change Handling

When a device rotates, Android may destroy and recreate the Activity. If important state is stored only in the Activity, search text, selected filters, or current results may be lost. In CineVault, the ViewModel stores UI-related state and exposes it through LiveData, so the recreated Activity can observe the existing state and redraw the screen consistently.

Android vs Flutter comparison

The Android implementation is the main project focus because it directly demonstrates MVVM using Java, Activities, ViewModel, LiveData, Repository, local data, and a prepared Retrofit layer. The Flutter prototype is useful as a small comparison because it shows that similar architectural goals can be achieved with Dart widgets and simple state management: the UI should display state, while data preparation and state changes should be kept outside the visual layout code. However, the Flutter prototype is limited to static/local data and is included only to support the architectural discussion, not to replace the Android MVVM implementation.

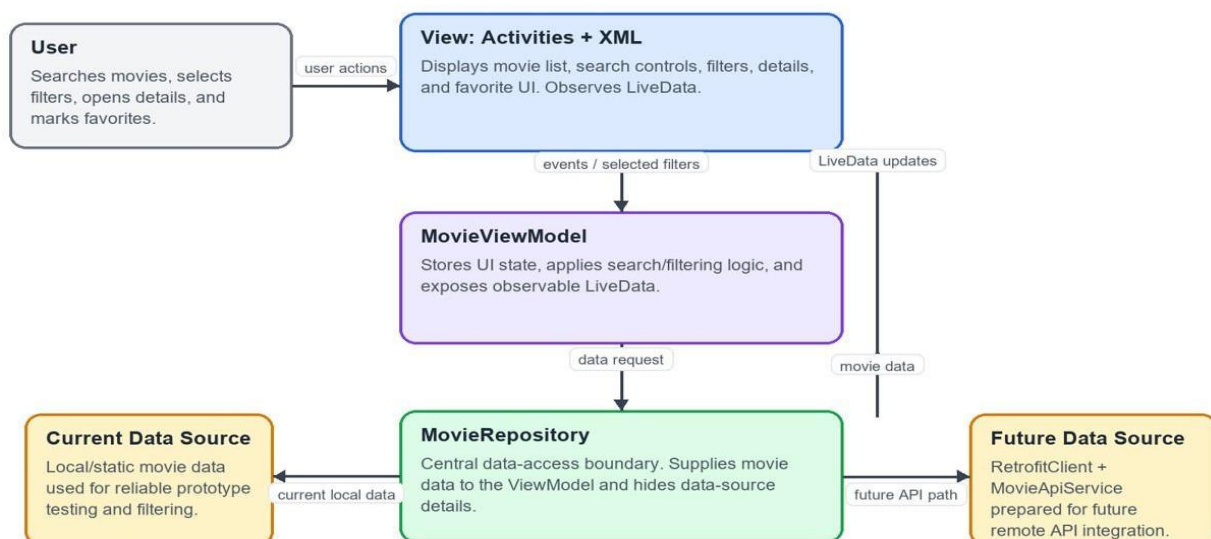


Figure 1: CineVault MVVM architecture.

3.3 Key Code Snippets & Breakdown:

The following two code screenshots should be inserted as evidence for the most important architectural decisions in CineVault. The screenshots should come from the actual Java implementation and should reflect the project's real architecture: Activities/XML, ViewModel, LiveData, Repository, local movie data, and the prepared Retrofit layer.

Code Screenshot 1: MovieViewModel filtering logic

```

public void filterByGenre(String genre) {
    selectedGenre.setValue(genre);
    activeGenre = genre;

    if (genre.equalsIgnoreCase("All Genres")) {
        showAllMovies();
        return;
    }

    currentMode = FilterMode.GENRE;
    uiState.setValue(new MovieUiState(
        genre.equalsIgnoreCase("Action"),
        genre.equalsIgnoreCase("Comedy"),
        genre.equalsIgnoreCase("Drama"),
        genre.equalsIgnoreCase("Kids"),
        filterBySelectedGenre("Action", genre),
        filterBySelectedGenre("Comedy", genre),
        filterBySelectedGenre("Drama", genre),
        filterBySelectedGenre("Kids", genre)
    ));
}

public void filterByYear(String year) {
    selectedYear.setValue(year);
    activeYear = year;

    if (year.equalsIgnoreCase("All Years")) {
        showAllMovies();
        return;
    }

    currentMode = FilterMode.YEAR;
    uiState.setValue(new MovieUiState(
        true,
        true,
        true,
        true,
        getMoviesByGenreAndYear("Action", year),
        getMoviesByGenreAndYear("Comedy", year),
        getMoviesByGenreAndYear("Drama", year),
        getMoviesByGenreAndYear("Kids", year)
    ));
}

public void searchByTitle(String title) {
    String normalized = title == null ? "" : title.trim();
    activeSearch = normalized;
    searchQuery.setValue(normalized);

    if (normalized.isEmpty()) {
        showAllMovies();
        return;
    }

    currentMode = FilterMode.SEARCH;
    uiState.setValue(new MovieUiState(
        true,
        true,
        true,
        true,
        searchMoviesByTitleAndGenre(normalized, "Action"),
        searchMoviesByTitleAndGenre(normalized, "Comedy"),
        searchMoviesByTitleAndGenre(normalized, "Drama"),
        searchMoviesByTitleAndGenre(normalized, "Kids")
    ));
}

```

Figure 2: MovieViewModel search and filtering logic.

Breakdown:

- The ViewModel is responsible for processing user input and producing the filtered movie list state.
- The Activity remains responsible for rendering the user interface and observing LiveData updates.
- Keeping search and filtering logic inside the ViewModel improves separation of concerns, enhances testability, and prevents UI classes from becoming overloaded with business logic.

Code Screenshot 2: MovieRepository local data

```
public Call<MovieResponse> getPopularMovies(String apiKey) {  
    return apiService.getPopularMovies(apiKey);  
}  
  
public List<Movie> getLocalMovies() {  
    List<Movie> movies = new ArrayList<>();  
  
    movies.add(new Movie("John Wick 4", "Action", 8.1, 2023, "Action movie", R.drawable.johnwick));  
    movies.add(new Movie("Mission: Impossible", "Action", 7.8, 2023, "Spy action movie", R.drawable.mission));  
    movies.add(new Movie("The Dark Knight", "Action", 9.0, 2008, "Batman crime action movie", R.drawable.darkknight));  
    movies.add(new Movie("Mad Max: Fury Road", "Action", 8.1, 2015, "Post-apocalyptic action movie", R.drawable.madmax));  
}
```

Figure 3: MovieRepository data-access boundary.

Breakdown:

- The Repository centralizes data access so the ViewModel does not directly depend on local arrays, files, or networking classes.
- The current prototype uses local data for stable search and filtering tests.
- The Retrofit layer is future-ready infrastructure and should be described as planned API integration, not as a fully active backend feature.

4 Discussion and Conclusion

4.1 Summary of Findings:

CineVault prototype proves that MVVM is a great approach to developing a sustainable movie discovery app. The primary observation is that the separation of concerns positively impacts code quality; Activities continue to be in charge of the interface's visual representation and handling user interactions, ViewModels take care of the state of the screens and filtering logic, while the Repository takes care of the data source.

The use of LiveData implies reactivity of the User Interface. If something changes, the ViewModel will update the observable collection of movies, and the Activity will

refresh the UI based on the changes made in the list. This will help make UI lighter and prevent duplicated code.

Another valuable aspect of the design that will come in handy in the future is the Repository pattern itself. Since the app currently uses data stored locally, the testing phase becomes easy and consistent. Meanwhile, the use of Retrofit prepares the app for using APIs in the future, and nothing else should be altered.

Strengths:

- Clear separation between UI rendering, UI-state logic, and data access.
- ViewModel and LiveData provide lifecycle-aware state management.
- Search and filtering operations take place within the ViewModel instead of the Activity.
- Repository Pattern makes it ready for any changes in the database in the future.
- Local database makes it more reliable to show the prototype. The Flutter prototype adds a useful but limited cross-platform comparison.

Weaknesses:

- The current Android prototype uses local movie data rather than a live API, so remote-data behavior is future integration rather than completed functionality.
- MVVM adds extra classes and structure, which can feel heavier for very small applications.
- The Flutter comparison is intentionally small and should be treated as supporting context rather than equal implementation depth.

4.2 Lessons Learned

The project has confirmed the impact of architecture on the quality of source code and developers' productivity. The separation of Activity, ViewModel, and Repository allowed for better understanding of these parts of the application, since UI-related changes did not affect data access and vice versa. Changes in data sources did not require placement of logic related to data sources into the Activity.

- MVVM provides value to applications with changing UI state like the search input state, filter selections, favorite movies, and selected movie details.
- It is right to place search and filter logics in ViewModel since it produces UI state and remains available even after configuration changes.

- LiveData appears to be a reasonable observable-state library for a Java implementation of Android MVVM.
- Using Repository Pattern makes local data and future API data interchangeable for ViewModel.
- Use Retrofit through Repository pattern so as not to introduce network code in Activities.
- Flutter implements another set of technologies but the same architectural approach: UI components should reflect the application state.

In conclusion, CineVault demonstrates that MVVM is not only a theoretical pattern but a practical approach for organizing a real mobile prototype. The architecture supports maintainability, lifecycle-aware state, testable filtering logic, and future backend integration. The strongest improvement remaining is to complete the planned Retrofit API connection and compare the behavior of local and remote data sources while preserving the same MVVM boundaries.

4.3 Team Contributions:

Team Member	Responsibilities	Deliverables
Tuleen Rimawi	Android Details Screen, Favorites Screen, Flutter Details Screen, flutter Favorites Screen CineVault MVVM architecture & Flutter state-management implementation	Report Sections 2.1, 3.2, 3.3, 4,5 & (Microsoft, 2012) executive summary
Walaa Jaradat	Android Movie Model, Movie Repository, Local Movie Data, Retrofit Setup (RetrofitClient & API Structure), Flutter Movie Model & Local Data	Report Sections 2.1 , 3.1 + final report polish
Masa Inabi	Splash Screen, Home UI, Android SearchView, RecyclerView Design, Flutter Home UI	Report Sections 1.1 , 2.1
Aya Jiaan	Filter by genre and year Search by name	Report Sections 1.2 , 2.1

Mona Omar	Android ViewModel & LiveData, Observer Handling, State Management, Flutter Provider/ChangeNotifier & the Presentation	Report Sections 1.3 , 2.1
-----------	---	---------------------------

5. References

- Android Developers. (2026, june 15). *Guide to app architecture*. Retrieved from Android Developers: <https://developer.android.com/topic/architecture>
- Android Developers. (2026, june 15). *ViewModel overview*. Retrieved from Android Developers: <https://developer.android.com/topic/libraries/architecture/viewmodel>
- Android Developers. (n.d.). *Guide to app architecture*. Retrieved from Android Developers: <https://developer.android.com/topic/architecture>
- Developers, G. (2024). *Guide to app architecture*. Retrieved from <https://developer.android.com/topic/architecture>
- Fowler, M. (2004). *Presentation Model*. Retrieved from <https://martinfowler.com/eaDev/PresentationModel.html>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
- Google. (2026, 4 14). Guide to App Architecture. *Guide to App Architecture*, 1. Retrieved from <https://developer.android.com/topic/architecture>
- Google. (2026, 5 18). LiveData Overview. *LiveData Overview*, 1. Retrieved from <https://developer.android.com/topic/libraries/architecture/livedata>
- Google. (2026, 5 29). State Management. *State Management*, 1. Retrieved from <http://docs.flutter.dev/data-and-backend/state-mgmt/simple>
- Microsoft. (2012). *The MVVM Pattern*. Retrieved from <https://learn.microsoft.com/en-us/previous-versions/xamarin/xamarin-forms/enterprise-application-patterns/mvvm>
- Square. (n.d.). Retrieved from Retrofit: <https://github.com/square/retrofit>
- Torunoglu, Z. (2024). *MVVM in Android*. Turkey: Medium.
- Trivedi, V. (2024). *How ViewModel Works Internally: A Deep Dive*. medium.